

◆ CS EDUCATION ◆

🔍

트랜잭션: ACID

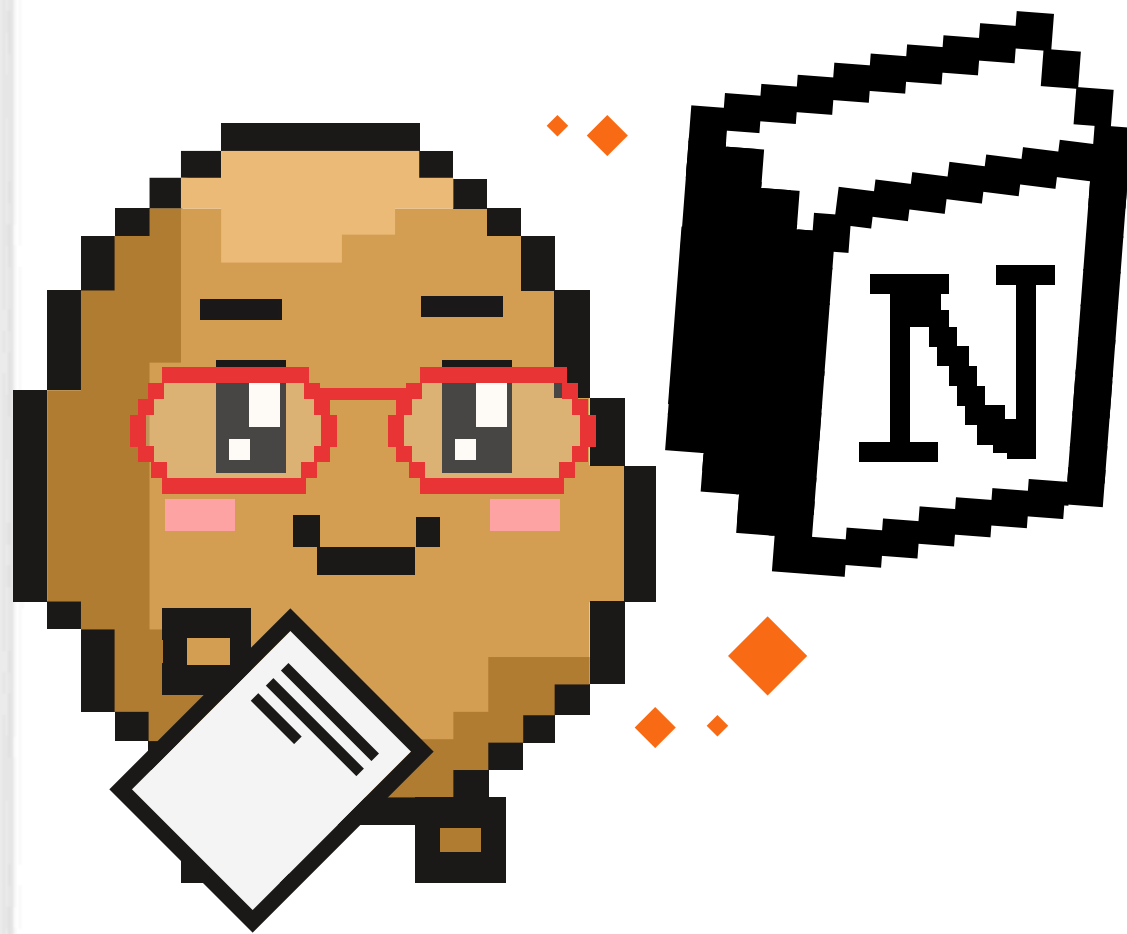
26.02.06

12기 교육팀 우재원

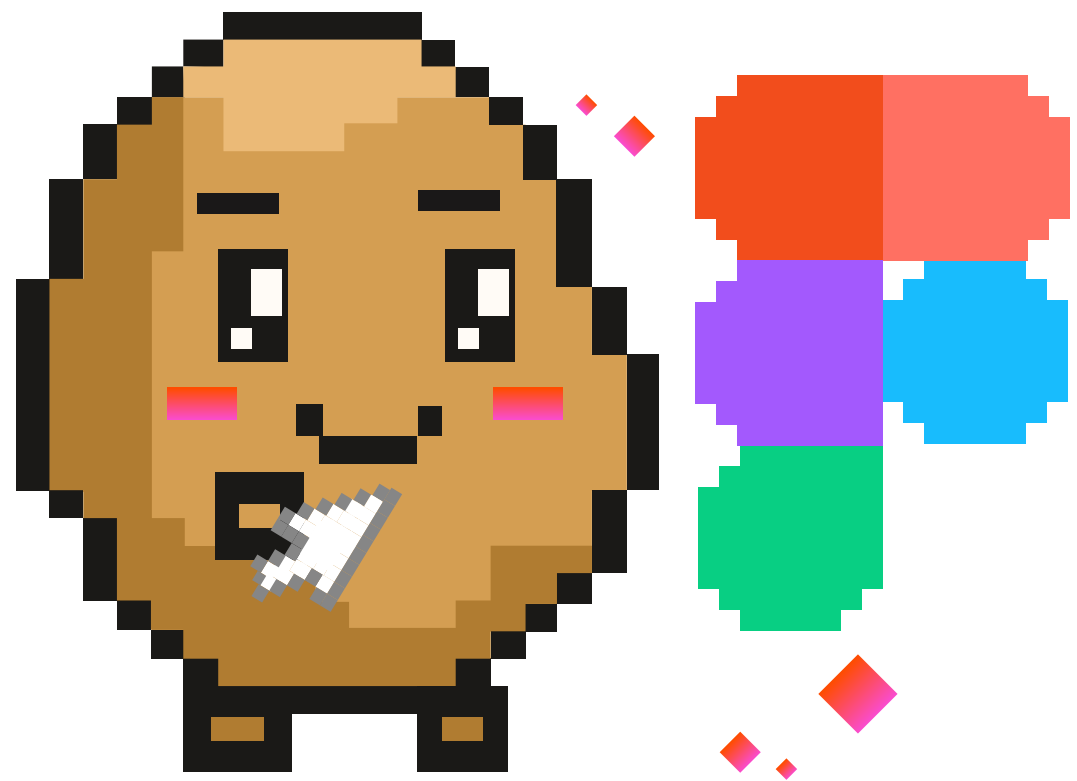
교육 목표

1. 트랜잭션의 정의와 제어(Commit/Rollback) 이해
2. 트랜잭션의 4대 속성(ACID) 파악
3. 격리성 보장을 위한 DB Lock의 동작 원리

트랜잭션의 필요성



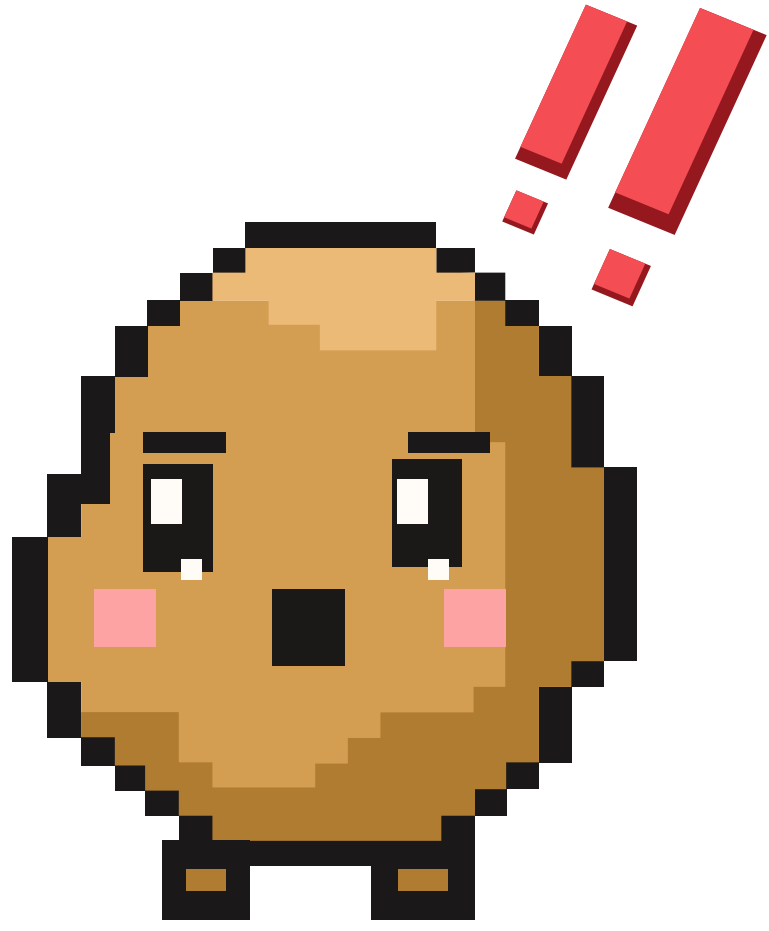
A→B 5천원 전송	기획 감자 (A)	디자인 감자 (B)
1. 기존 계좌 잔고	10000	10000
2. A 잔고 - 5천원	10000 -5000	10000
3. B 잔고 + 5천원	5000	10000 +5000



트랜잭션의 필요성



A→B 5천원 전송	기획 감자 (A)	디자인 감자 (B)
1. 기존 계좌 잔고	10000	10000
2. A 잔고 - 5천원	10000 -5000	10000
3. B 잔고 + 5천원	5000	10000+5000
4. 3 도중 에러 발생 ?	5000	10000 + 5000



트랜잭션 : 더 이상 나눌 수 없는 최소 단위



트랜잭션(Transaction)의 정의

DB 의 상태를 변화시키는, '논리적인 작업의 단위'

Commit과 Rollback



Commit, Rollback 코드 예시

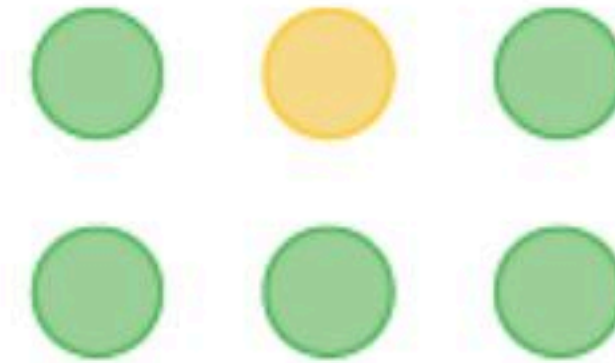
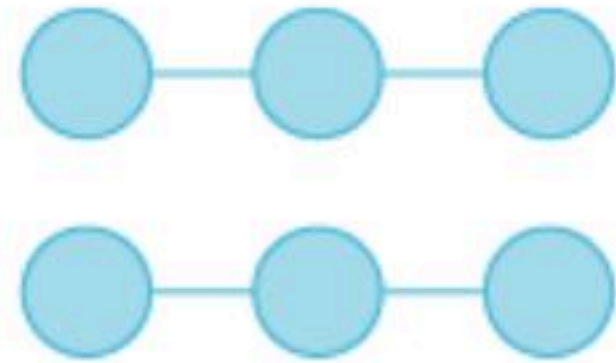
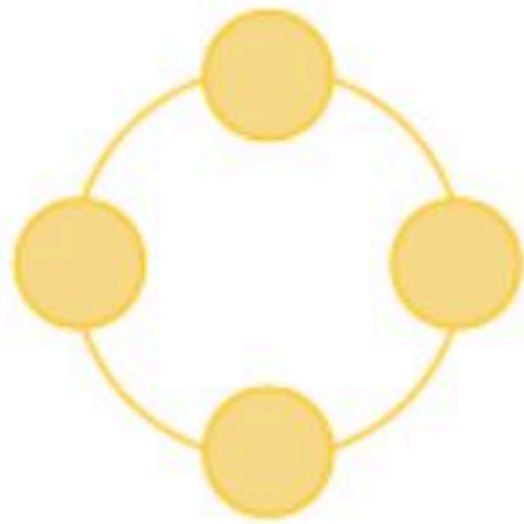
```
try:
    start_transaction()
    # 출금
    account_a.balance -= 5000
    # 입금
    account_b.balance += 5000
    commit()
except Exception:
    rollback()
```

All or Nothing 원칙을 코드로 구현

성공 시 COMMIT,
실패 시 ROLLBACK

→ 데이터 안전성 보장

ACID란?



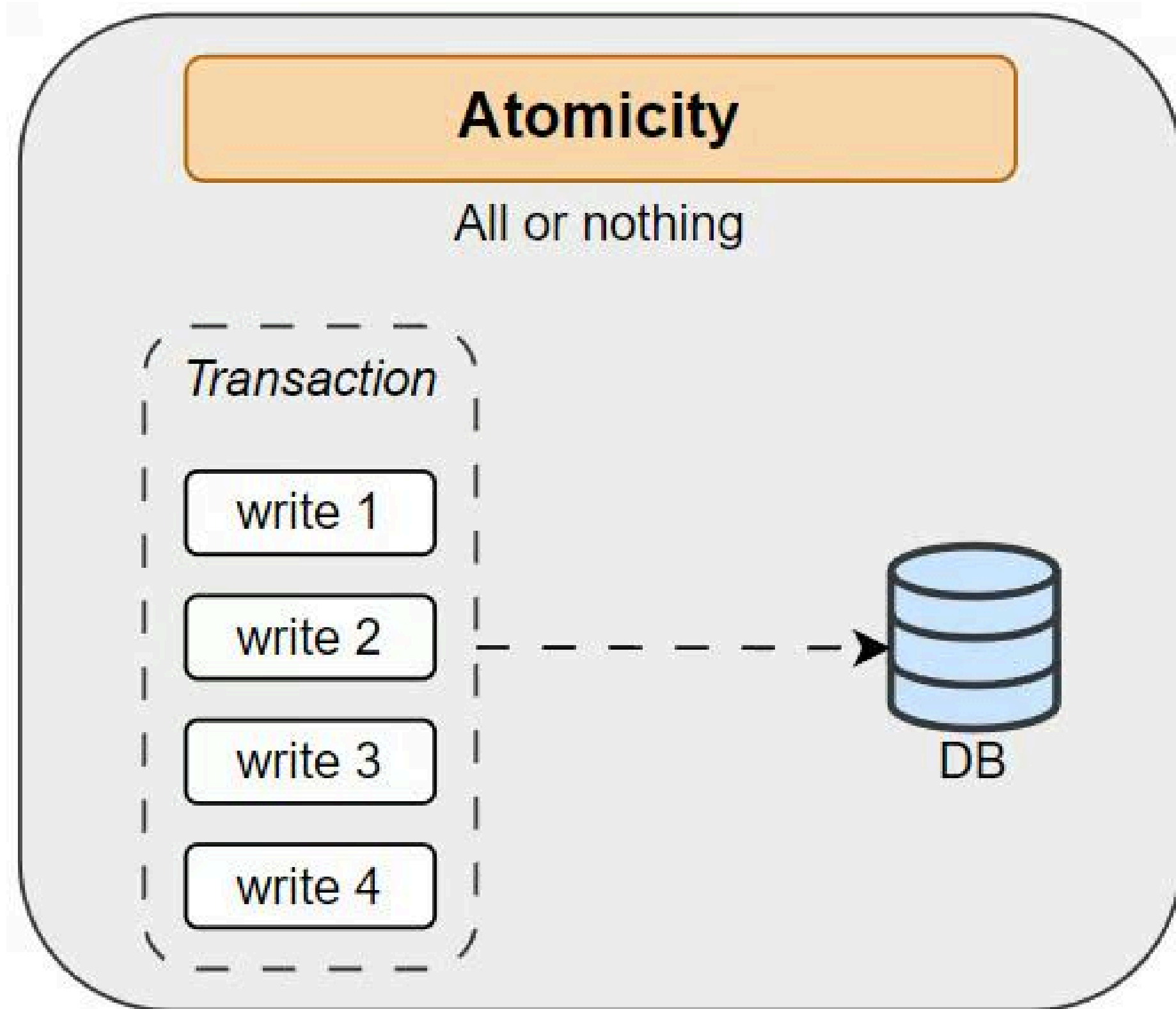
Atomicity

Consistency

Isolation

Durability

Atomicity (원자성)

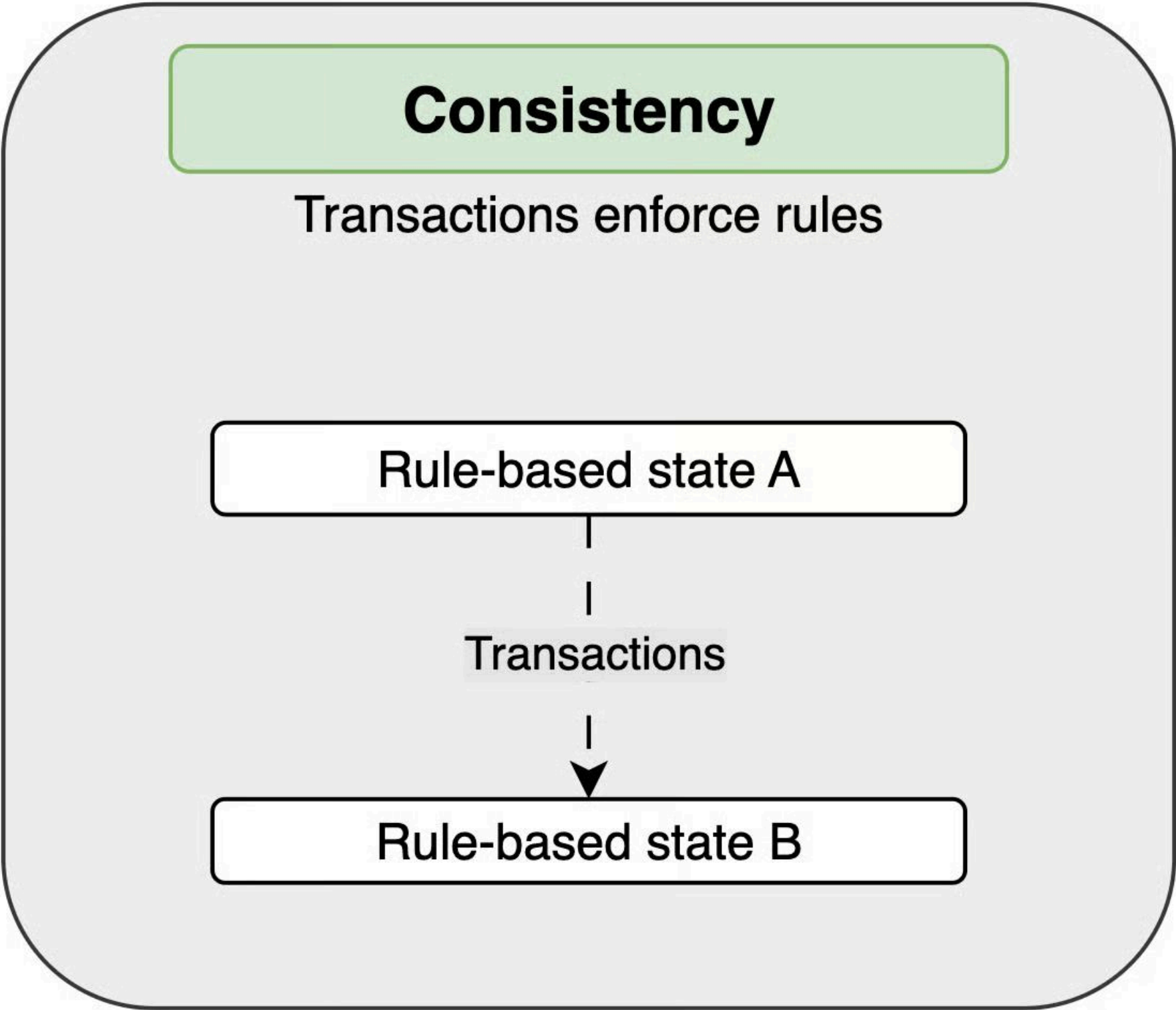


모든 작업(write 1~4)이 묶여서

함께 실행되거나,
아예 실행되지 않거나

둘 중 하나만 가능!

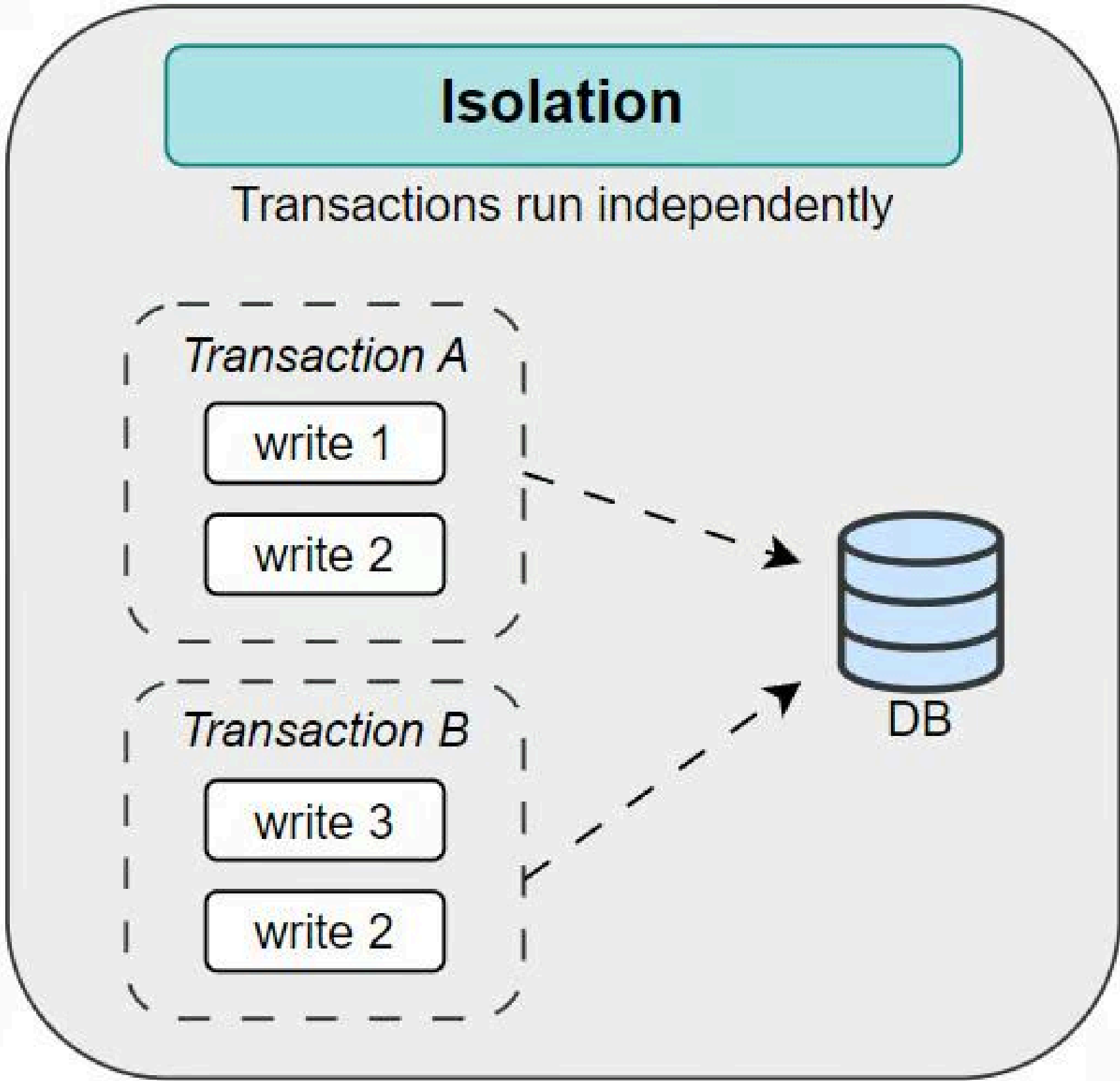
Consistency (일관성)



규칙을 지킨 상태 A 에서,
규칙을 지킨 상태 B 로만 전환

규칙 위반 불가!

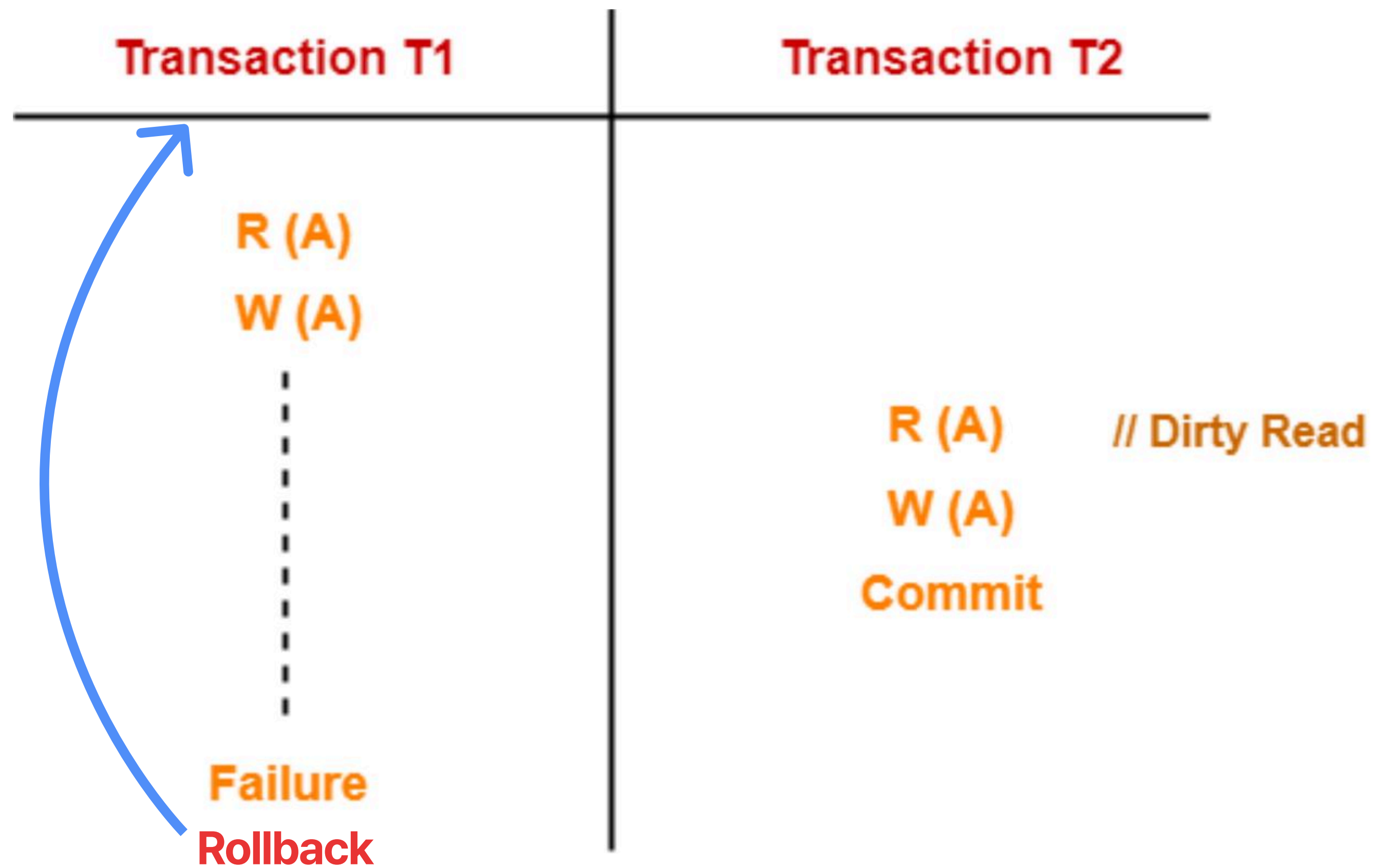
Isolation (격리성/고립성)



여러 트랜잭션이 **동시 실행되어도**
마치 혼자 실행되는 것처럼

서로 간섭 없음!

격리성 - Lock



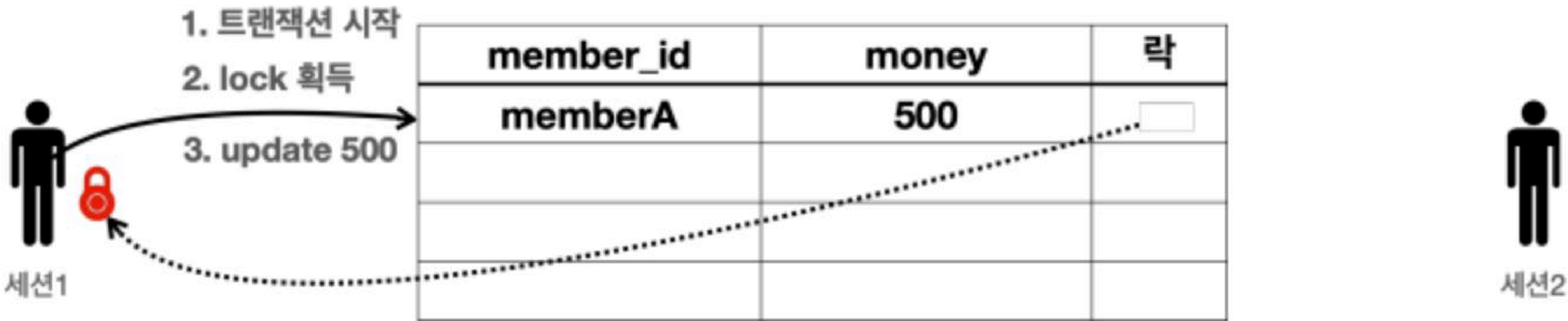
T1 수정 (커밋 X) → T2 가 읽음

: Dirty Read (더티 리드)

T1 롤백 시, T2 데이터는 무효

결과: 데이터 불일치 문제 발생

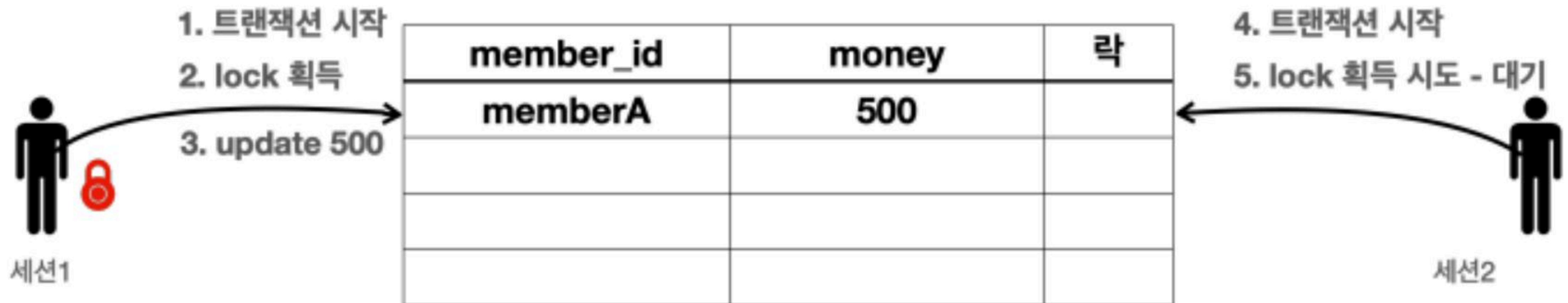
DB Lock (변경) - 획득



세션 1: 데이터를 수정하려는 첫 번째 사용자

세션 2: 데이터를 수정하려는 두 번째 사용자

DB Lock (변경) - 대기



세션 1 : 데이터를 수정하려는 첫 번째 사용자

세션 2 : 데이터를 수정하려는 두 번째 사용자

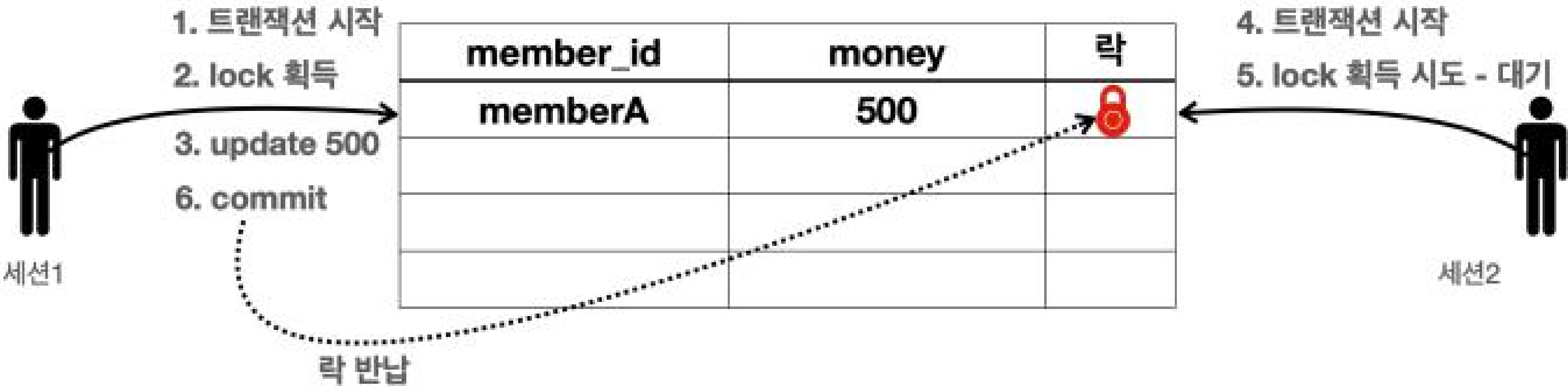
Lock 타임아웃

세션 1이 Lock을 잡은 채 퇴근하면,
세션 2는 무한정 대기 ?

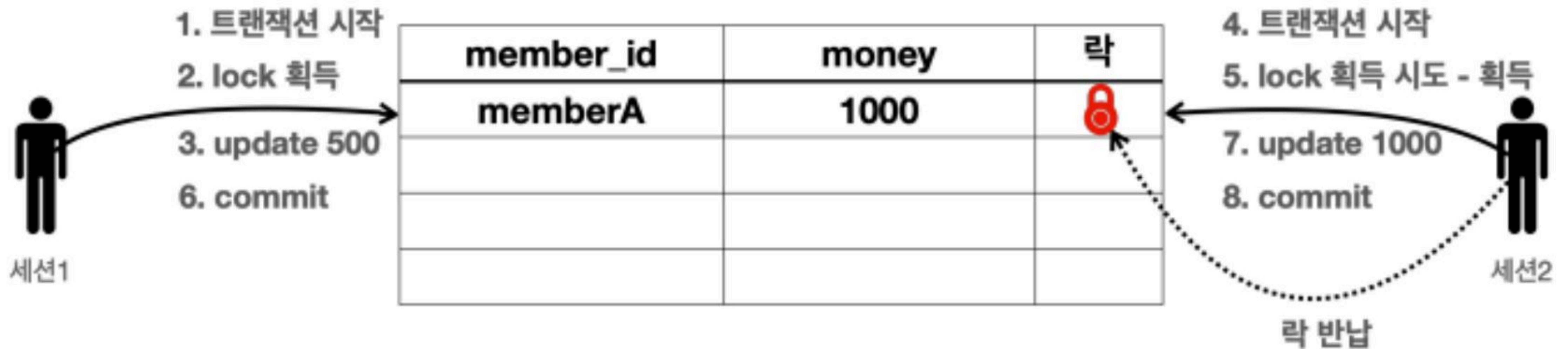
X. 무한 대기 : 시스템에 치명적
→ Lock 타임아웃
= 특정 시간 기다린 후, Timeout 에러

```
# SET LOCK_TIMEOUT <milliseconds>  
SET LOCK_TIMEOUT 60000; # 60 초
```


DB Lock (변경) - 세션 1 commit, 반납



DB Lock (변경) - 세션 2 Lock 획득, commit, 반납



DB Lock (조회)

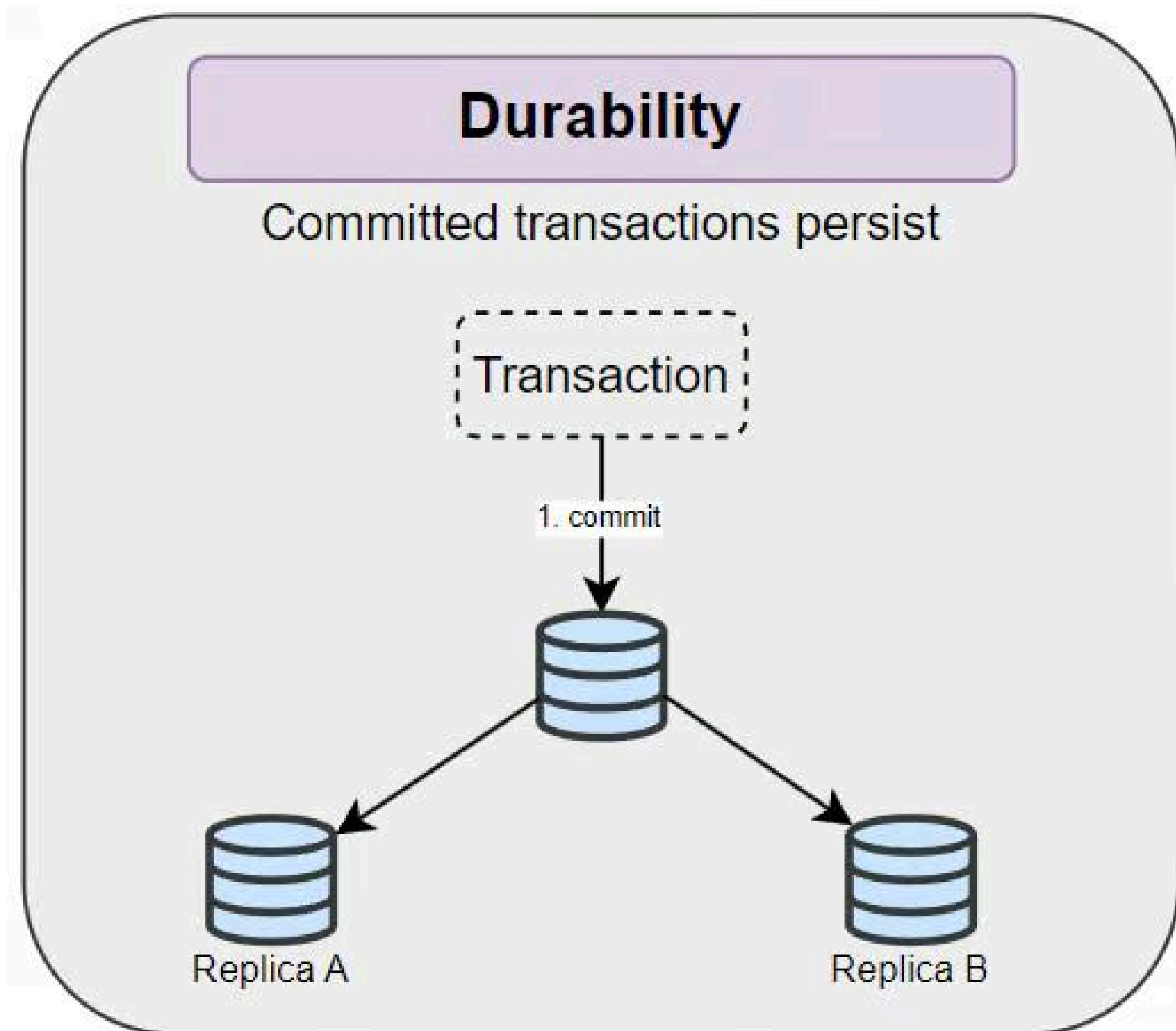
select 데이터 조회 : 보통 lock 사용 x. 조회하는 동안 데이터 변동을 꼭 막아야 하면?

세션 1 수정 중이어도,
세션 2는 조회 가능

select for update :
조회 시점에 강제로 lock

```
set autocommit false;  
select * from member where member_id='memberA' for update; -- 락 획득!
```


Durability (지속성)



한 번 COMMIT 되면,
여러 곳에 안전하게 저장

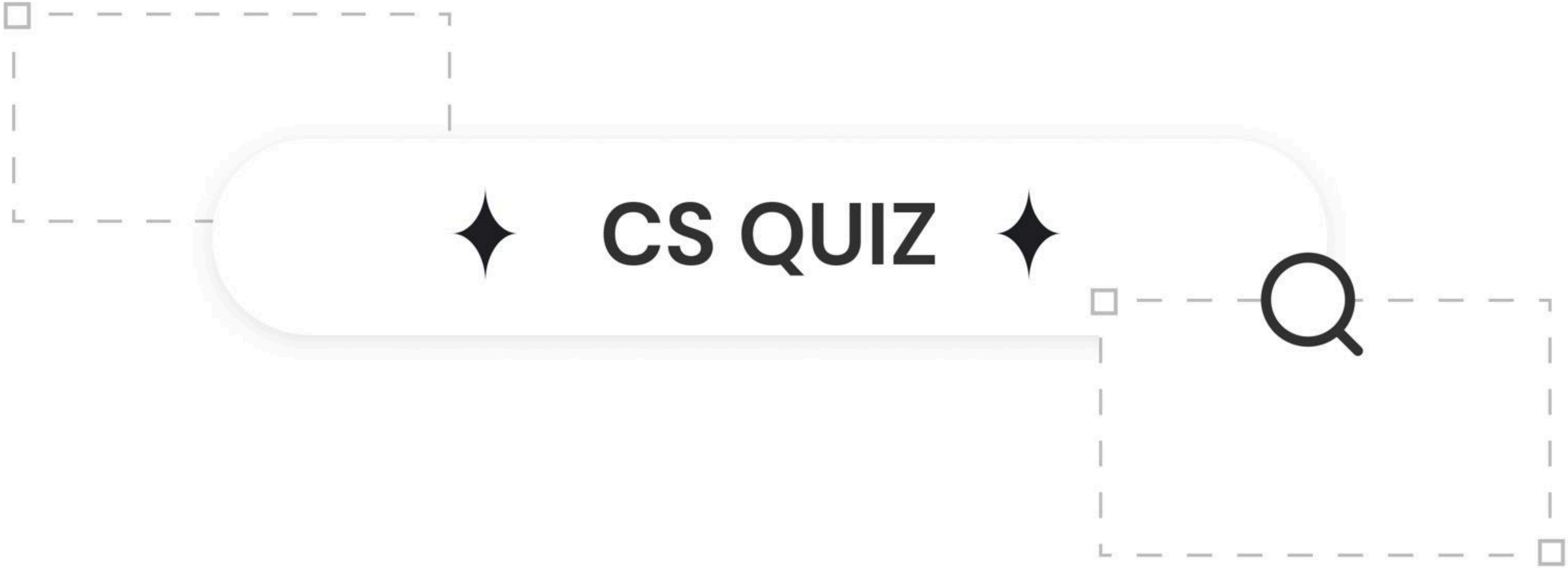
영구 유지!

요약

트랜잭션 : DB 의 상태를 변화시키는 **논리적 작업 단위**
(성공: Commit, 실패: Rollback)

ACID : 원자성, 일관성, 격리성, 지속성 (**4대 속성**)

동시성 제어 (**Lock**) : 격리성을 위해 **데이터 변경/조회** 시 Lock 활용



Q.1

코테이토 CS 퀴즈 사이트에서 사용자가 문제를 풀고 '정답 제출'을 눌렀을 때, 점수 저장 과정 중 오류가 발생하면 저장 이전 상태로 되돌려 데이터가 꼬이지 않도록 해야 한다.

이때 데이터를 되돌리는 데 사용되는 트랜잭션 제어 명령어는 무엇일까요?
(주관식)



A.1

코테이토 CS 퀴즈 사이트에서 사용자가 문제를 풀고 '정답 제출'을 눌렀을 때, 점수 저장 과정 중 오류가 발생하면 저장 이전 상태로 되돌려 데이터가 꼬이지 않도록 해야 한다.

이때 데이터를 되돌리는 데 사용되는 트랜잭션 제어 명령어는 무엇일까요?
(주관식)

rollback (롤 백)

문제를 맞히면 여러 업데이트가 하나의 트랜잭션으로 묶여서 수행되므로, 이 과정에서 하나라도 실패하면 중간까지 저장된 데이터가 남으면 안 되기 때문에 트랜잭션을 원래 상태로 되감기해야 함

Q.2

Commit 명령어의 역할로 가장 적절한 것은?

- ① 트랜잭션을 강제로 종료한다.
- ② 변경 사항을 취소하고 이전 상태로 되돌린다.
- ③ 변경 결과를 데이터베이스에 영구적으로 반영한다.
- ④ 락 (Lock) 을 획득한다.



A.2

Commit 명령어의 역할로 가장 적절한 것은?

- ① 트랜잭션을 강제로 종료한다. → 커밋에 대한 설명 X
- ② 변경 사항을 취소하고 이전 상태로 되돌린다. → 롤백
- ③ 변경 결과를 데이터베이스에 영구적으로 반영한다.
- ④ 락 (Lock) 을 획득한다. → 커밋에 대한 설명 X

Q.3

ACID에서 'I'가 의미하는 속성의 이름은? (주관식)



A.3

ACID에서 'I'가 의미하는 속성의 이름은? (주관식)

isolation (격리성, 고립성)

하나의 트랜잭션이 완료되기 전까지,
다른 트랜잭션이 그 데이터를 조회하거나 수정하지 못하게 하여
데이터의 일관성을 유지하는 것

Q.4

다음 설명이 옳바르면 O, 그렇지 않으면 X를 고르시오.

Commit이 완료된 데이터는
시스템 장애가 발생해도 유지되어야 한다.



A.4

다음 설명이 올바른면 O, 그렇지 않으면 X를 고르시오.

Commit이 완료된 데이터는
시스템 장애가 발생해도 유지되어야 한다.

정답: O

이는 Durability(지속성)의 정의로,
한 번 커밋된 데이터는 영구적으로 저장되어야 한다.

Q.5

다음 중 ACID의 성질을 가장 올바르게 설명한 것은?

- ① 원자성(Atomicity) : 여러 트랜잭션이 동시에 실행될 때, 서로 간섭하지 않는 것.
- ② 일관성(Consistency) : 트랜잭션이 커밋된 이후에도, 데이터가 제약조건을 유지하는 것.
- ③ 격리성(Isolation) : 시스템 장애 발생 시, 트랜잭션이 복구 가능한 상태로 저장되는 것.
- ④ 지속성(Durability) : 트랜잭션이 부분적으로만 실행되어도, 결과가 저장되는 것.

A.5

다음 중 ACID의 성질을 가장 올바르게 설명한 것은?

- ① 원자성(Atomicity) : 여러 트랜잭션이 동시에 실행될 때, 서로 간섭하지 않는 것.
→ 격리성
- ② 일관성(Consistency) : 트랜잭션이 커밋된 이후에도, 데이터가 제약조건을 유지하는 것.
- ③ 격리성(Isolation) : 시스템 장애 발생 시, 트랜잭션이 복구 가능한 상태로 저장되는 것.
→ 엉뚱한 설명
- ④ 지속성(Durability) : 트랜잭션이 부분적으로만 실행되어도, 결과가 저장되는 것.
→ 원자성에 반하는 설명

Q.6

서비스 이벤트로 '선착순 1명'에게만 경품을 지급하는 로직을 만들었습니다.
이때 트랜잭션과 락을 제대로 설정하지 않으면 어떤 문제가 생길까요?

- ① 서버가 느려져서 아무도 경품을 못 받는다
- ② 원자성 때문에 당첨자가 0명이 된다.
- ③ 격리성이 깨져서 1명만 당첨되어야 하는데, 동시에 여러 명이 당첨된 것으로 처리될 수 있다.
- ④ 지속성이 부족해서 당첨자 명단이 다음날 사라진다.

A.6

서비스 이벤트로 '선착순 1명'에게만 경품을 지급하는 로직을 만들었습니다.
이때 트랜잭션과 락을 제대로 설정하지 않으면 어떤 문제가 생길까요?

- ① 서버가 느려져서 아무도 경품을 못 받는다 → 트랜잭션과 상관 없음
- ② 원자성 때문에 당첨자가 0명이 된다. → 원자성 : all or nothing
- ③ 격리성이 깨져서 1명만 당첨되어야 하는데, 동시에 여러 명이 당첨된 것으로 처리될 수 있다.
- ④ 지속성이 부족해서 당첨자 명단이 다음날 사라진다. → 지속성: 성공한 데이터를 지킴

Q.7

일반적인 SELECT 문과 달리 조회 시점부터 배타적 락을 걸 수 있는 SQL 구문은?

- ① SELECT LOCK
- ② SELECT FOR UPDATE
- ③ SELECT WITH LOCK
- ④ SELECT EXCLUSIVE



A.7

일반적인 SELECT 문과 달리 조회 시점부터 배타적 락을 걸 수 있는 SQL 구문은?

- ① SELECT LOCK
- ② SELECT FOR UPDATE
- ③ SELECT WITH LOCK
- ④ SELECT EXCLUSIVE

일반적인 SELECT : 락 사용 X

조회한 데이터를 바탕으로 중요한 계산을
수행하는 동안 값이 변하면 안 되는 경우,
SELECT...FOR UPDATE 구문을 사용해
조회 시점부터 강제로 락을 걸 수 있음

Q.8

아래 상황에서 발생한 문제를 방지하기 위해 설정해야 할 개념으로 가장 적절한 것은? (주관식)

관리자 A는 코테이토 회원 정보를 수정하기 위해 트랜잭션을 시작하고 update 쿼리를 실행했다. 그러던 중 급한일이 생겨서 트랜잭션을 종료하지 않은 채 자리를 비우게 되었다.

그 사이 관리자 B가 동일한 회원 레코드를 수정하려 했고, 해당 레코드에 대한 락이 해제되지 않아 요청이 오랜 시간 대기 상태에 빠지게 되었다.



A.8

아래 상황에서 발생한 문제를 방지하기 위해 설정해야 할 개념으로 가장 적절한 것은? (주관식) **락 타임아웃 (Lock Timeout)**

관리자 A는 코테이토 회원 정보를 수정하기 위해 트랜잭션을 시작하고 update 쿼리를 실행했다. 그러던 중 급한일이 생겨서 트랜잭션을 종료하지 않은 채 자리를 비우게 되었다.

그 사이 관리자 B가 동일한 회원 레코드를 수정하려 했고, 해당 레코드에 대한 락이 해제되지 않아 요청이 오랜 시간 대기 상태에 빠지게 되었다.

DB 나 시스템에서 특정 자원에 대한 잠금(Lock)을 획득하려는 작업이 대기 상태에 빠졌을 때, 정해진 시간 내에 락을 얻지 못하면 오류를 발생시키고 요청을 포기하는 메커니즘.

Q.9

데이터베이스 락(Lock)의 동작 원리에 대한 설명으로 올바른 것은 무엇인가요?

- ① 세션 1이 데이터를 수정하기 위해 락을 획득하면,
세션 2는 락이 반납될 때까지 대기해야 한다
- ② 데이터 조회(Select) 시에는 어떠한 경우에도 락을 사용할 수 없다.
- ③ 락을 획득하지 못한 세션은 즉시 에러를 발생시키고 종료된다.
- ④ 세션 1이 락을 잡고 퇴근하면 세션 2는 무한정 대기하게 된다.

A.9

데이터베이스 락(Lock)의 동작 원리에 대한 설명으로 올바른 것은 무엇인가요?

- ① 세션 1이 데이터를 수정하기 위해 락을 획득하면,
세션 2는 락이 반납될 때까지 대기해야 한다
- ② 데이터 조회(Select) 시에는 어떠한 경우에도 락을 사용할 수 없다.
→ 조회 시에도 강제로 락을 걸 수 있음
- ③ 락을 획득하지 못한 세션은 즉시 에러를 발생시키고 종료된다.
→ 락을 얻을 때까지 대기
- ④ 세션 1이 락을 잡고 퇴근하면 세션 2는 무한정 대기하게 된다.
→ 무한 대기로 인한
장애를 방지하기 위해 Lock Timeout

Q.10

세션 A가 특정 행에 대해 **SELECT FOR UPDATE** 를 실행하여 락을 획득했습니다.
이때 세션 B가 동일한 행에 대해 '일반 **SELECT**'(락 없음)를 실행하면 어떻게 될까요?

- ① 세션 A가 커밋할 때까지 세션 B는 무한 대기한다.
- ② 세션 B는 락과 상관없이 즉시 데이터를 조회할 수 있다.
- ③ 데이터베이스가 '접근 거부' 에러를 발생시킨다.
- ④ 세션 B가 조회하는 순간 세션 A의 락이 자동으로 해제된다.



A.10

세션 A가 특정 행에 대해 **SELECT FOR UPDATE** 를 실행하여 락을 획득했습니다.
이때 세션 B가 동일한 행에 대해 '일반 **SELECT**' (락 없음)를 실행하면 어떻게 될까요?

① 세션 A가 커밋할 때까지 세션 B는 무한 대기한다.

→ 수정 or 락 시도해야만 막음.

② 세션 B는 락과 상관없이 즉시 데이터를 조회할 수 있다.

③ 데이터베이스가 '접근 거부' 에러를 발생시킨다.

→ 에러가 아닌, 대기.

④ 세션 B가 조회하는 순간 세션 A의 락이 자동으로 해제된다.

→ 세션 A의 트랜잭션이 종료(Commit 또는 Rollback)될 때 해제.

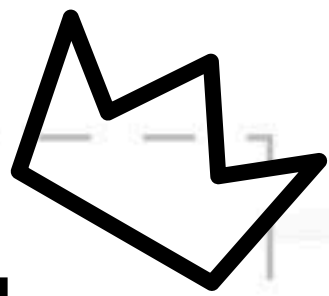
Q11

다음 상황에서 올바른 동작을 고르시오.

- 현정의 계좌에서 5,000원 출금
- 철웅의 계좌에 5,000원 입금
- 입금 시점에 네트워크 오류 발생

- ① 입금만 반영하고, 출금은 되돌린다.
- ② 이미 출금이 실행되었으므로 둘 다 그대로 둔다.
- ③ 출금만 반영하고, 나중에 수동으로 입금 처리한다.
- ④ 출금과 입금 모두 롤백되어, 두 계좌 모두 원래 잔액으로 돌아간다.

A11



다음 상황에서 올바른 동작을 고르시오.

- 현정의 계좌에서 5,000원 출금
- 철웅의 계좌에 5,000원 입금
- 입금 시점에 네트워크 오류 발생

① 입금만 반영하고, 출금은 되돌린다.

트랜잭션은 부분 반영 허용 X

② 이미 출금이 실행되었으므로 둘 다 그대로 둔다.

현정의 돈 5,000원 사라짐

③ 출금만 반영하고, 나중에 수동으로 입금 처리한다.

트랜잭션의 정상적인 동작 원리 X

④ 출금과 입금 모두 롤백되어, 두 계좌 모두 원래 잔액으로 돌아간다.



◆ CS EDUCATION ◆

🔍

감사합니다!

26.02.06

12기 교육팀 우재원

